

LEVEL 9

Chapter 10: Security

System Design Master Roadmap — 9 Years of Experience Edition

Compiled August 2026



Chapter 10: Security in System Design

← [Chapter 9: API Design](#) · [Index](#) · Next → [Chapter 11: Observability & Operations](#)

10.1 Why Security Belongs in Every HLD Answer, Not a Separate Round

Most candidates treat security as an afterthought bolted on if the interviewer explicitly asks. At Tier-1 and fintech companies specifically (Razorpay, PhonePe, Amazon), unprompted security awareness is an active positive signal, and its complete absence is a real deduction — Chapter 31's rubric allocates explicit points to it. The goal isn't to turn every answer into a security audit; it's to naturally mention the two or three security decisions that matter *for this specific design* as you go, the same way you'd mention caching or scaling.

10.2 AuthN, AuthZ, RBAC, ABAC

Authentication vs. authorization was covered in Chapter 1 — here's the authorization *model* layer, which is what actually gets asked about in HLD interviews for anything with permissions.

RBAC (Role-Based Access Control): users are assigned roles (`admin` , `merchant` , `customer`), and permissions are attached to roles, not individual users. Simple, auditable, and the right default for most systems — "can this role delete an order" is a small, reviewable table.

ABAC (Attribute-Based Access Control): access decisions are computed from attributes of the user, resource, and context (e.g., "a support agent can view an order only if they're assigned to that region AND the order is less than 90 days old AND it's during their shift hours"). More expressive, but harder to audit and reason about at a glance — reach for it when RBAC's static roles genuinely can't express your rules, not by default.

Interview question: "How would you design authorization for a marketplace where sellers should only manage their own listings, but admins can manage everything?"

Ideal senior answer: "RBAC as the base — `seller`, `admin` roles — plus a resource-ownership check layered on top for the seller role specifically: 'seller can edit a listing only if `listing.seller_id == requesting_user.id`.' That ownership check is really a lightweight ABAC rule sitting on top of an RBAC foundation, which is how most real systems end up looking — pure RBAC alone can't express 'your own resources only' without either an explosion of per-user roles or exactly this kind of attribute check."

10.3 OAuth2, JWT, API Keys — Applied

Covered mechanically in Chapter 1; the applied version: **OAuth2** for delegated third-party access and "login with X" flows; **JWT** for stateless service-to-service and API authentication where you control both ends; **API keys** for simpler machine-to-machine or partner integrations where you want an easily revocable, easily rate-limited, non-expiring-by-default credential (store hashed, like passwords — never log or return a raw API key after initial issuance).

10.4 Transport and Data Protection

TLS (encryption in transit): covered in Chapter 1 — every external-facing endpoint, non-negotiable. Internally, for regulated data (payments, PII), encrypt service-to-service traffic too (mutual TLS via a service mesh sidecar, e.g., Istio/Envoy, is the common pattern — you likely already have the EKS foundation for this).

Encryption at rest: data on disk (database volumes, S3 buckets, EBS volumes) encrypted so a stolen disk or leaked backup isn't directly readable. On AWS, this is largely a configuration decision (`RDS` encryption, `S3` default encryption) backed by **KMS (Key Management Service)** for key management — know that KMS separates "who can use a key to encrypt/decrypt" from "who can read the encrypted data," giving you a second access-control layer beyond the database's own permissions.

Secrets management: already covered in Chapter 8.4 — never in code/config, always in a secrets manager with scoped IAM access and rotation.

Password hashing: never store plaintext or reversibly-encrypted passwords. Use a slow, salted hashing algorithm designed for this (**bcrypt**, **scrypt**, or **argon2** — argon2 is the current best-practice recommendation) — these are deliberately slow (tunable "work factor") specifically to make brute-force attacks on stolen hash dumps expensive, unlike a fast general-purpose hash like SHA-256, which is the wrong tool here precisely because it's fast.

10.5 The Classic Web Vulnerabilities

Attack	What it is	Primary defense
SQL Injection	Untrusted input concatenated directly into a SQL query, letting an attacker inject their own SQL	Parameterized queries / prepared statements — always, never string-concatenate user input into SQL
XSS (Cross-Site Scripting)	Attacker injects malicious script into a page viewed by other users (e.g., via an unescaped comment field)	Escape/sanitize all user-generated content on output, use a strict Content-Security-Policy header
CSRF (Cross-Site Request Forgery)	A malicious site tricks a logged-in user's browser into making an unwanted request to your site, riding on their existing session cookie	CSRF tokens (a random token the form must include, unguessable by the attacker site) + <code>SameSite=Strict/Lax</code> cookies
SSRF (Server-Side Request Forgery)	Attacker tricks your <i>server</i> into making a request to an internal/unintended URL (e.g., a "fetch this image URL" feature abused to hit <code>http://169.254.169.254/</code> — the cloud metadata endpoint — to steal IAM credentials)	Strict allow-lists for any server-initiated outbound request to user-supplied URLs; block requests to internal/link-local IP ranges
DDoS	Overwhelming a system with traffic (often from many distributed sources) to deny service to legitimate users	Rate limiting, a WAF , CDN absorbing traffic at the edge, and a managed DDoS protection service (AWS Shield)

WAF (Web Application Firewall): sits in front of your application (often at the CDN/load balancer layer — AWS WAF integrates with CloudFront/ALB) and filters malicious traffic patterns (known SQL injection signatures, XSS payloads, bot traffic) before they reach your application, using rule sets you configure or manage.

Interview question: "Your app has a feature that lets a user upload an avatar from a URL. What security risk does this create, and how do you mitigate it?"

Ideal senior answer: "This is a textbook SSRF vector — a malicious user could supply `http://169.254.169.254/latest/meta-data/iam/security-credentials/` on AWS and potentially exfiltrate the instance's IAM credentials, or scan your internal network for other services. I'd mitigate with an allow-list of acceptable protocols/domains where possible, explicitly block requests to private/link-local IP ranges (169.254.0.0/16, 10.0.0.0/8, etc.) and `localhost`, and if the fetch happens server-side, run it from an isolated network segment with no route to internal services or the metadata endpoint at all — defense in depth, not just a single check."

10.6 Zero Trust, Audit Logging, PII Protection

Zero Trust: the principle that no request is trusted by default just because it originates "inside the network perimeter" — every request is authenticated and authorized on its own merits, regardless of source. Practically: service-to-service calls inside your VPC still carry and verify identity (mutual TLS, service-mesh-enforced policies) rather than assuming "it's internal, so it's safe." This matters increasingly as microservice counts grow — the old "hard shell, soft inside" perimeter model breaks down once there are hundreds of internal services, any one of which could be compromised.

Audit logging: an immutable record of *who did what, when* for sensitive actions (who viewed this customer's data, who approved this refund, who changed this permission) — required for compliance (PCI-DSS, SOC2, GDPR-adjacent regulations) and essential for incident investigation. Audit logs should be append-only and shipped somewhere the acting user/service can't tamper with after the fact.

PII (Personally Identifiable Information) protection: minimize what you collect and store in the first place; encrypt what you must store; mask/redact PII in logs (a genuinely common real-world bug — logging a full request body that happens to contain a card number or password); apply the principle of least privilege so only services/people who need PII access have it; and know the regional regulatory context relevant to your target companies — GDPR-style data residency and consent requirements are directly relevant for India/UAE-based companies serving or storing data related to EU users, and India's DPDP Act and UAE data protection regulations are increasingly relevant to mention if you're interviewing at companies operating in those markets specifically.

10.7 How Security Should Appear in Your Architecture Diagram

Don't bolt security on as an afterthought bullet point at the end — show it structurally:

- **TLS termination point** marked explicitly (usually at LB/CDN edge, with a note if you re-encrypt internally).
 - **API Gateway** shown as the auth/rate-limiting choke point, not implied.
 - A distinct **WAF** box in front of public-facing entry points for anything at meaningful scale or handling payments.
 - **Secrets Manager/KMS** shown as a dependency of services that need credentials, not omitted.
 - For fintech-adjacent designs: an explicit boundary/zone drawn around PCI-scope components (anything touching raw card data), since minimizing that scope is itself a real architectural decision companies make deliberately.
-

Chapter 10 Interview Drill

1. Explain the difference between RBAC and ABAC with an example only ABAC can naturally express.
2. Why is bcrypt/argon2 preferred over SHA-256 for password hashing, given SHA-256 is technically "more secure" in a cryptographic sense?
3. Walk through an SSRF attack scenario and its mitigation, end to end.
4. What's the difference between encryption in transit and at rest, and where does each apply in a typical 3-tier architecture?
5. Explain Zero Trust in one sentence, and why it matters more as microservice count grows.

Next → [Chapter 11: Observability & Operations](#) — logging, metrics, tracing, SLIs/SLOs/SLAs, and disaster recovery.